Ahmedabad University

ENR497: B.Tech. Engg. Project

School of Engineering & Applied Science

(Winter 2026)

Ahmedabad University

Agentic Cloud Cluster

Final Project Presentation

Sarthak Siddhpura
Mentor: Professor Sanjay Chaudhary

Jan 2026 - April 2026

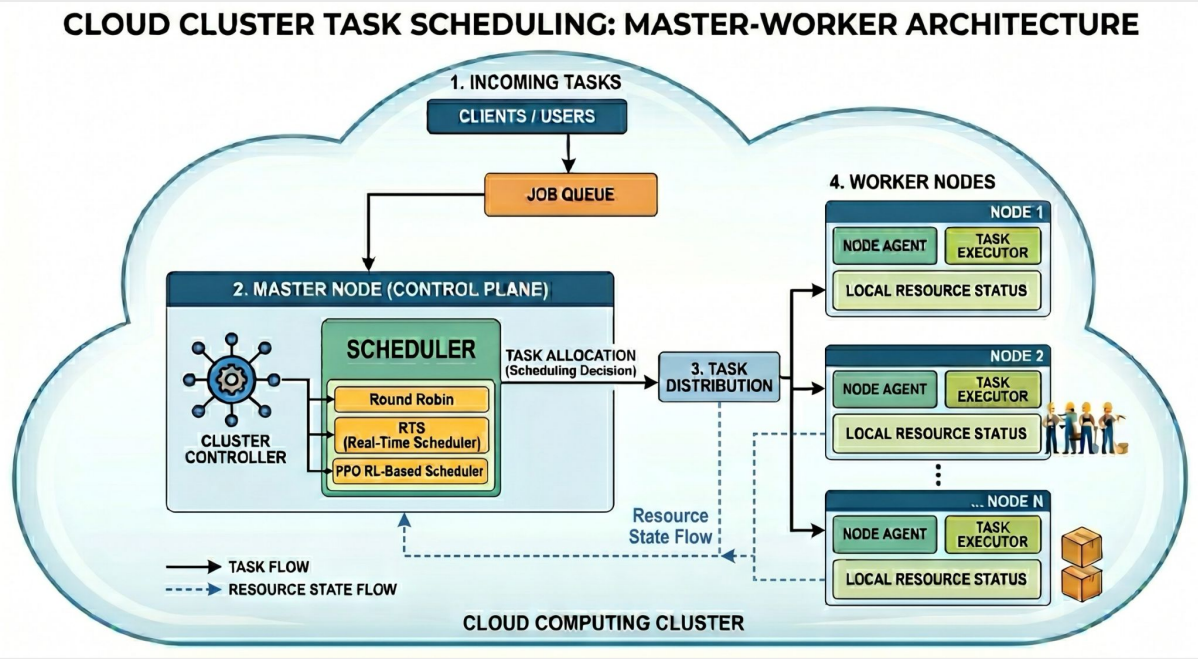
Ahmedabad University

Outline of Presentation

- Motivation
- Literature survey
- Overall objectives of the project
- Main outcomes of the project
- Methodology
- Progress and final results
- Novelty in the project
- Gantt chart for the project
- Who will benefit from your work ?

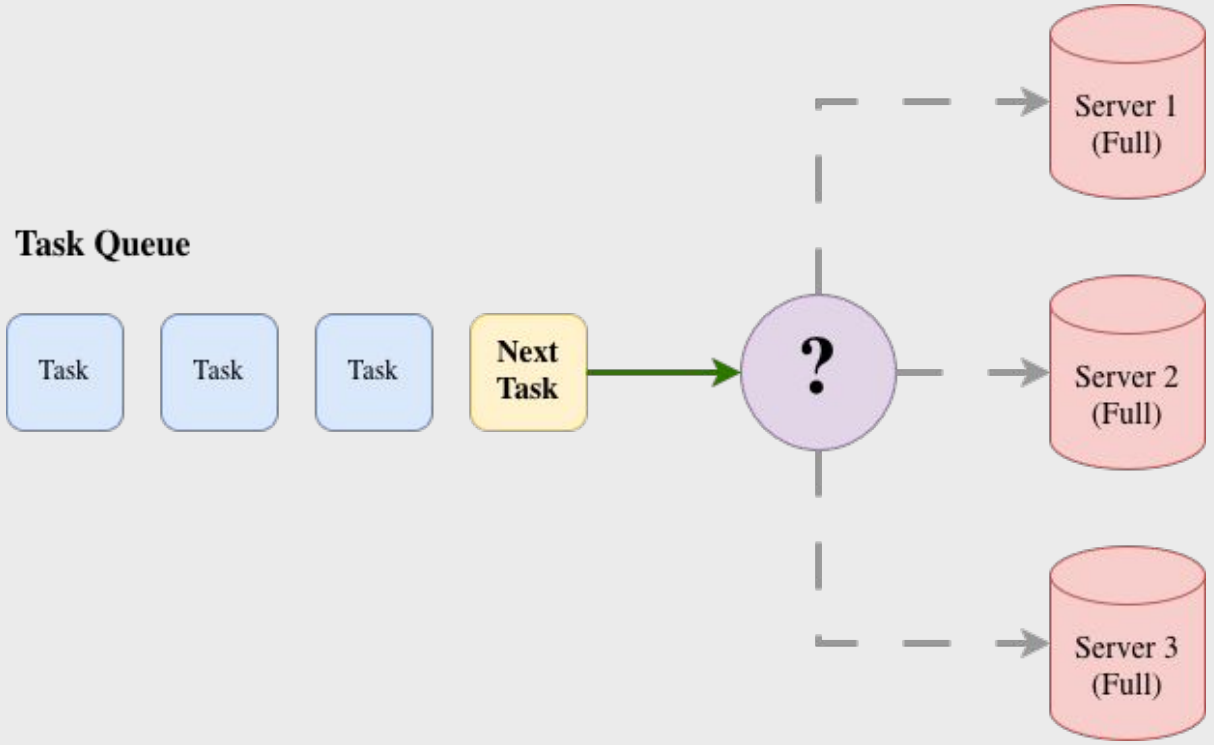
Agentic Cloud Cluster

- A distributed task execution system built in Go.
- Master-worker architecture coordinating Docker-based jobs.
- Task placement decisions enhanced with Proximal Policy Optimization (PPO).
- Clean separation between worker execution logic and the scheduling algorithm.



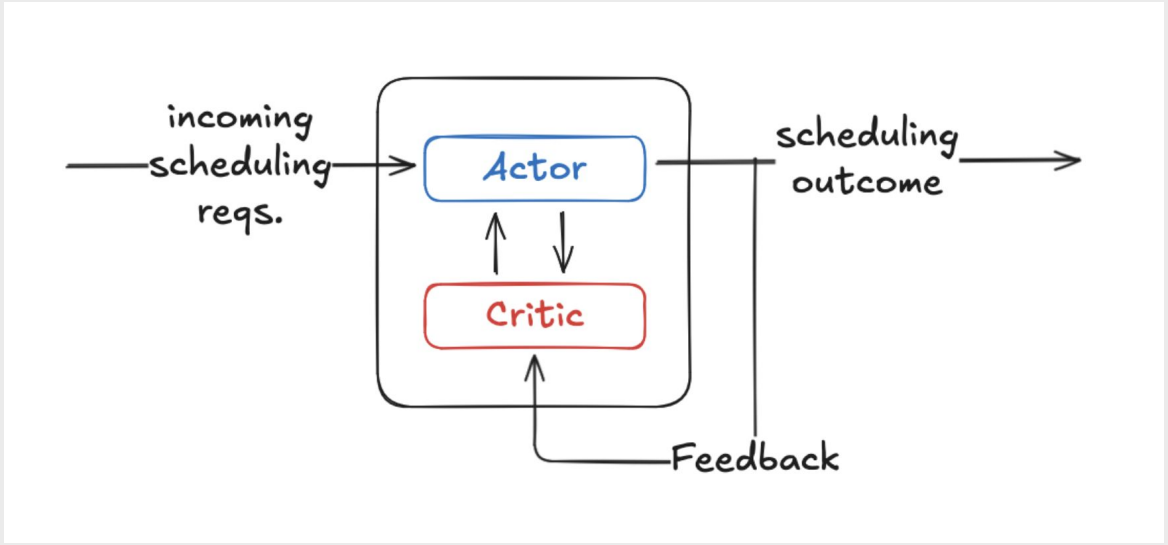
Motivation

- Simple scheduling rules (e.g., Round-Robin) fail to account for worker capacity or task urgency.
- Heuristic rules are hard to tune for mixed workloads (CPU-heavy vs. memory-heavy).
- Heterogeneous workers mean the "optimal" choice changes dynamically every few seconds.



Why Reinforcement Learning?

- **Scheduling** is a repeated decision problem modifying future cluster state.
- **State:** Current task requirements and worker resources.
- **Action:** The selected worker node.
- **Reward:** Feedback on placement feasibility, delay risk, and cluster balance.



Literature survey

- <Survey of similar work already existing (done by others industry/researchers)>

Evolution of Agentic Scheduling (Q-Tables to PPO)

- **Q-Tables**: Unsuitable due to massive, continuous state spaces in cluster environments.
- **Deep Q-Networks (DQN):** Difficult to handle dynamic action sets and masking invalid workers.
- **Proximal Policy Optimization (PPO):** Directly learns a policy, supports action masks, and uses stable clipped updates.

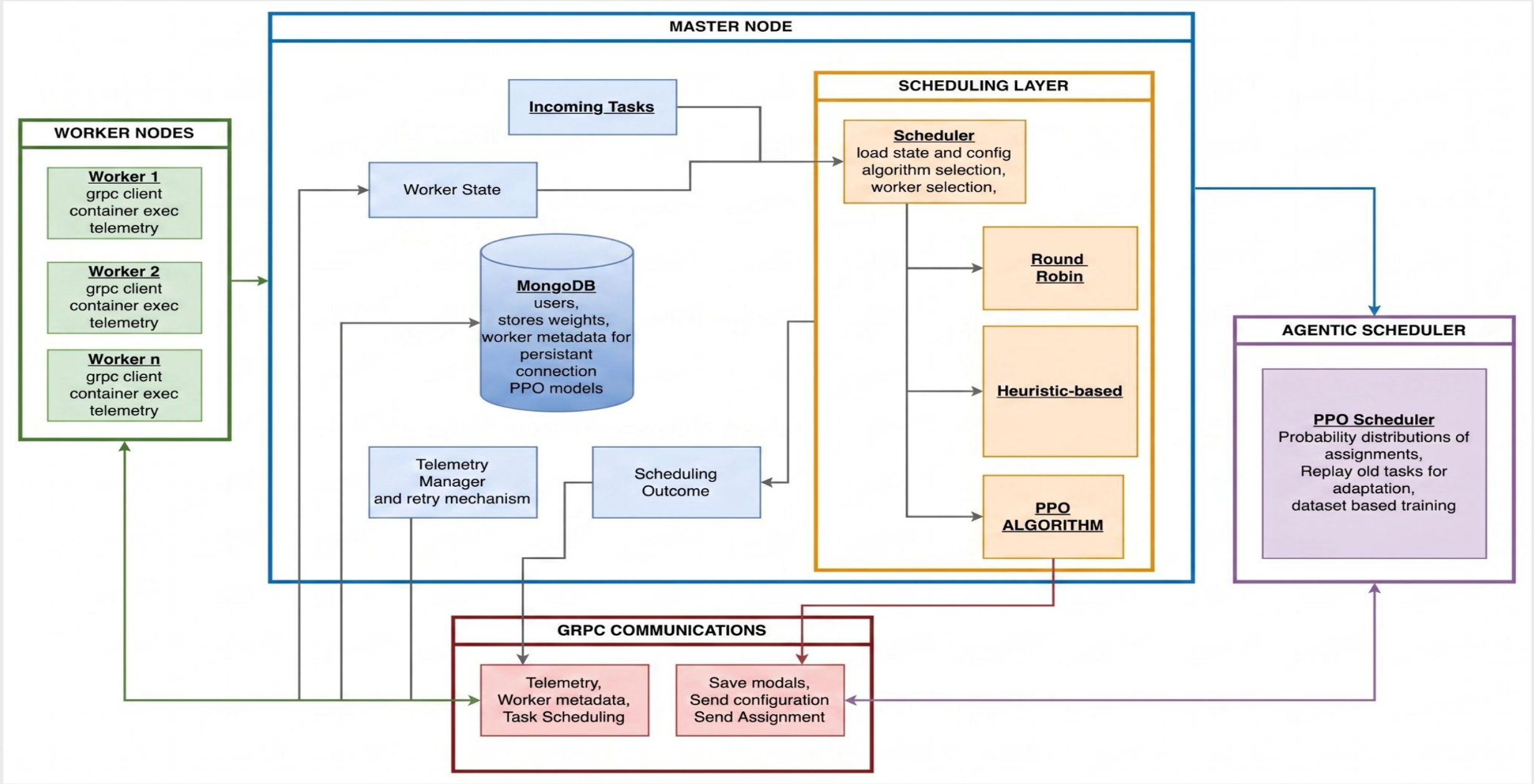
Relevant Tools and Technologies

- **Go:** Powers the highly concurrent master-worker control plane.
- **Docker:** Provides isolated, repeatable container execution environments.
- **gRPC & Protocol Buffers:** Ensures rigid, typed, and high-performance communication.
- **MongoDB:** Acts as the persistent storage layer for tasks, workers, and metadata.
- **Python/PyTorch:** Drives the decoupled PPO scheduling service.

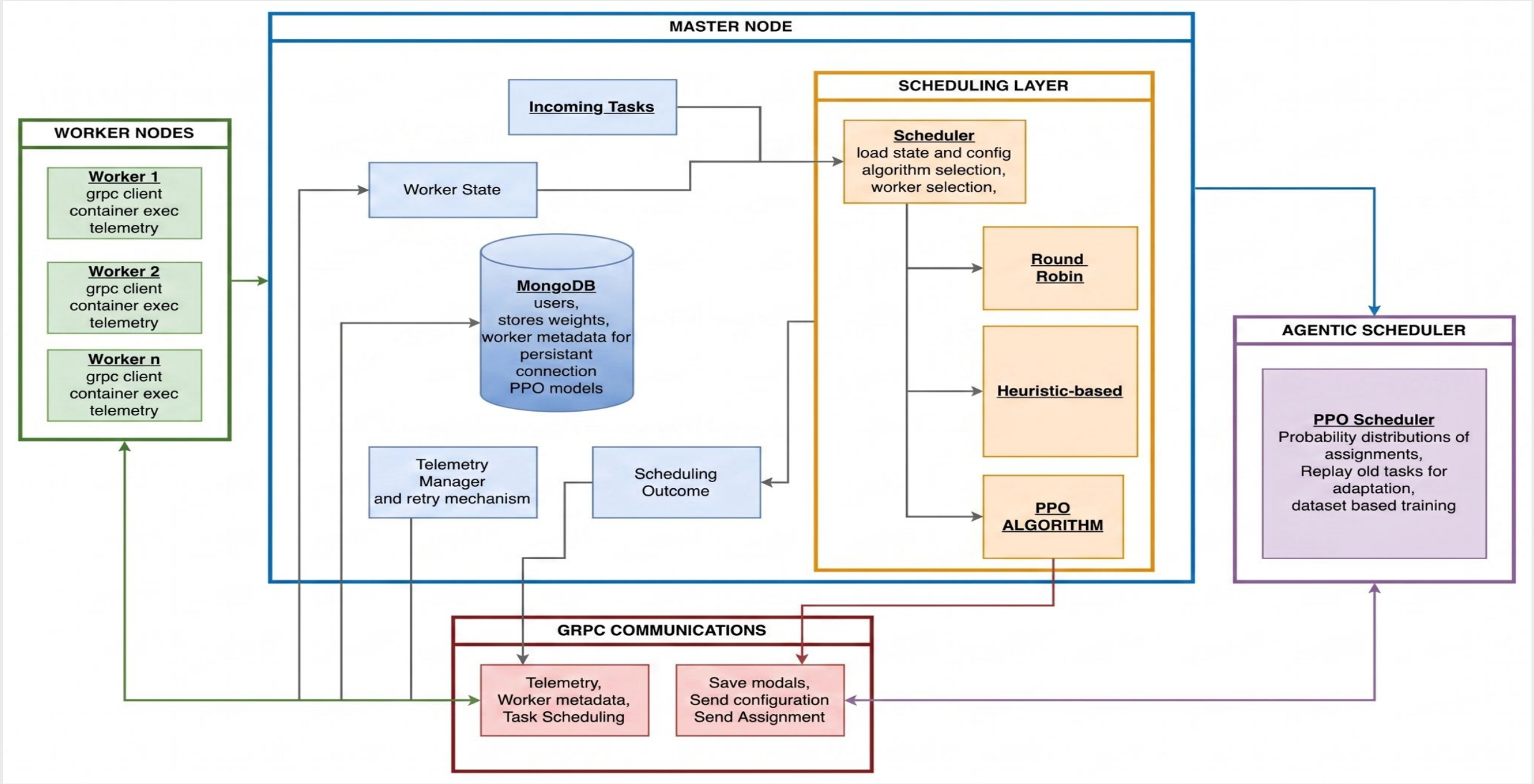
Objectives of the project

- Build a functional Go master-worker execution cluster.
- Implement worker registration, heartbeats, and resource tracking.
- Create a clean scheduling interface supporting both baselines and PPO.
- Train the PPO agent on realistic cluster traces with a comprehensive reward function.
- Conduct end-to-end benchmark campaigns to analyze system behavior.

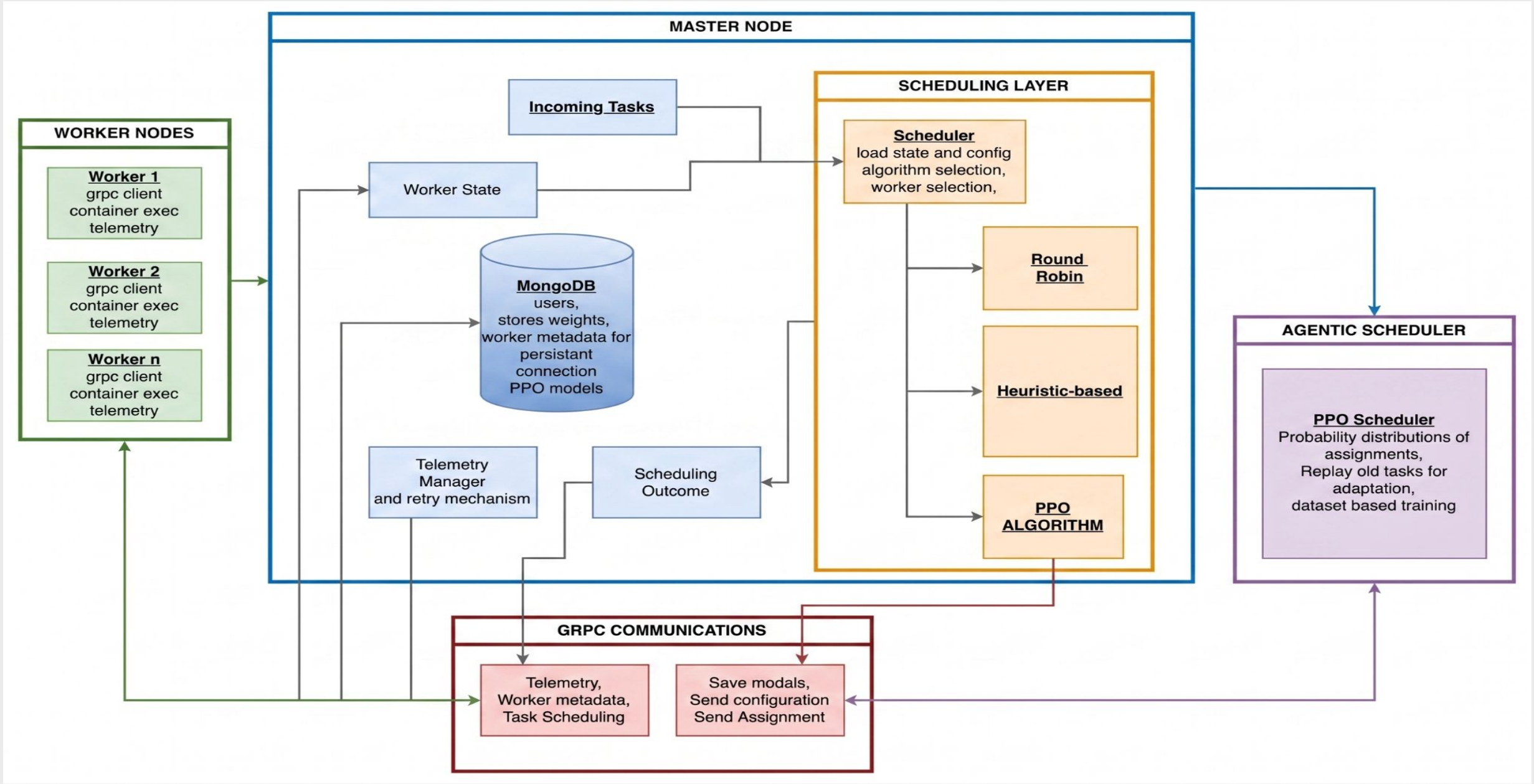
High-Level Architecture



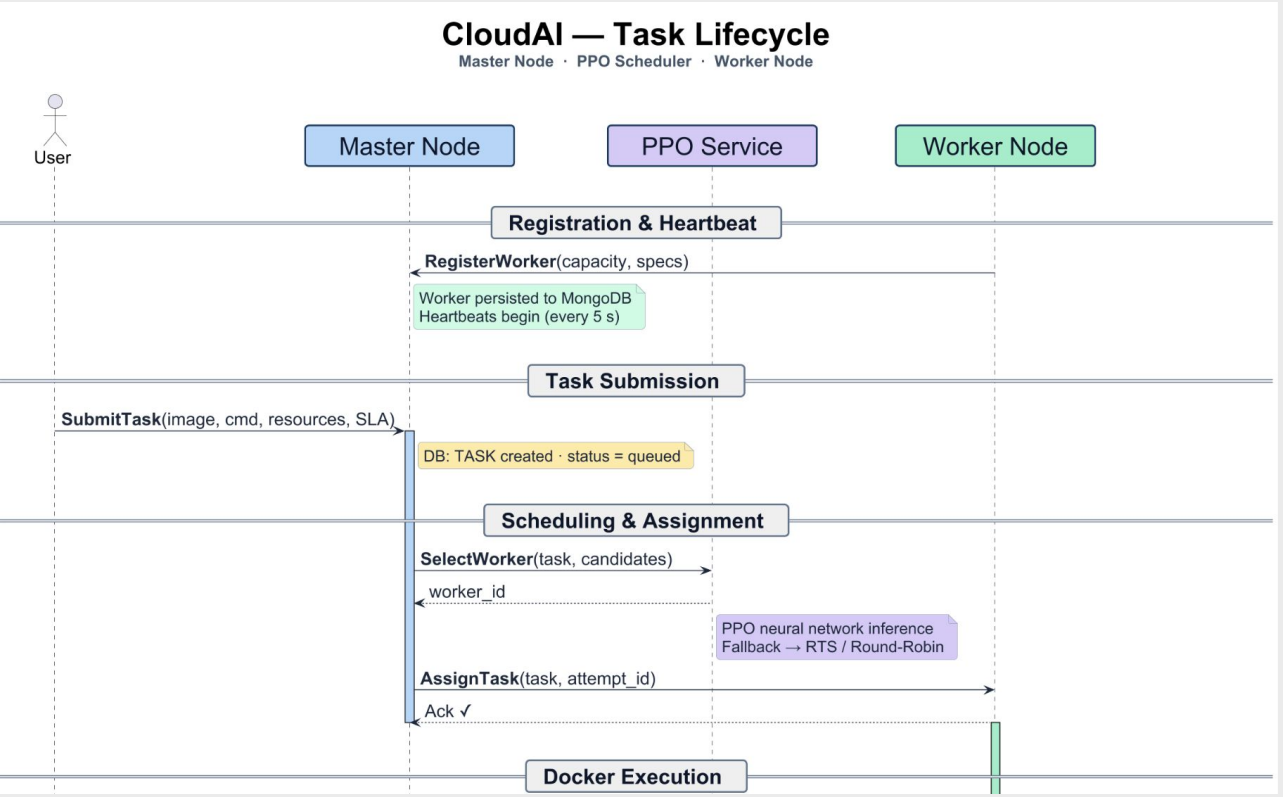
High-Level Architecture: Master Node

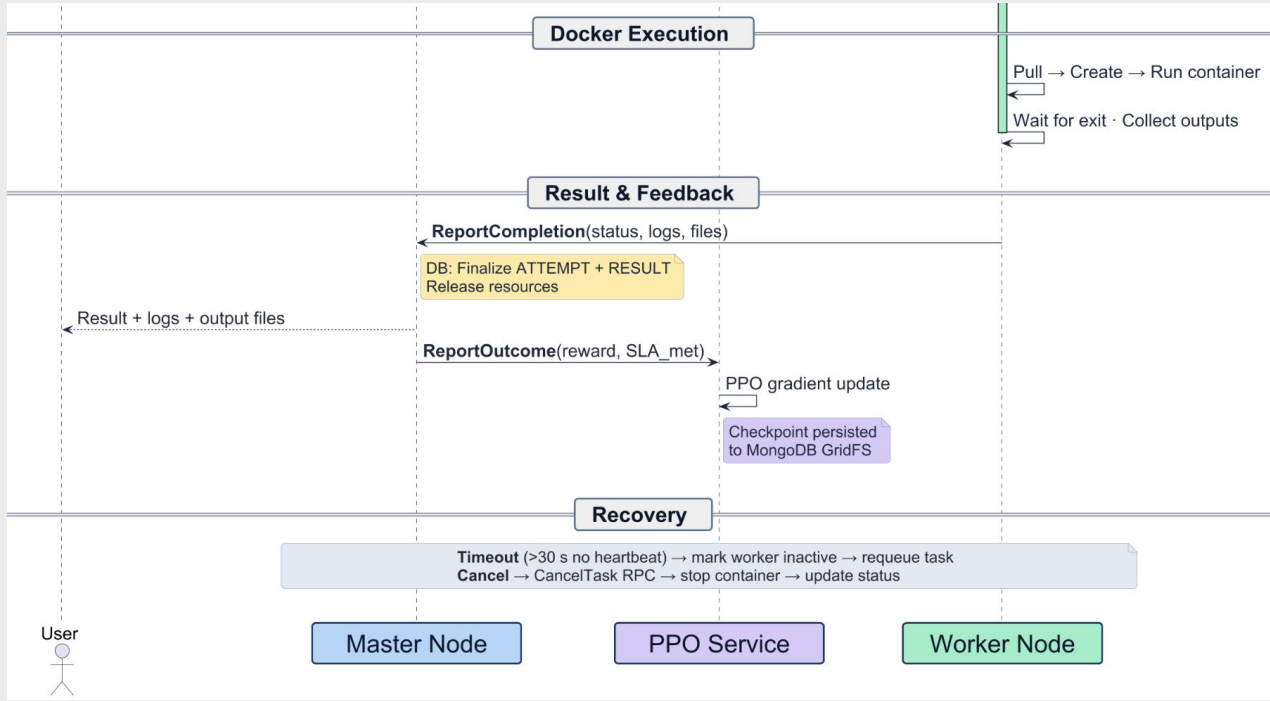


High-Level Architecture: Worker Nodes

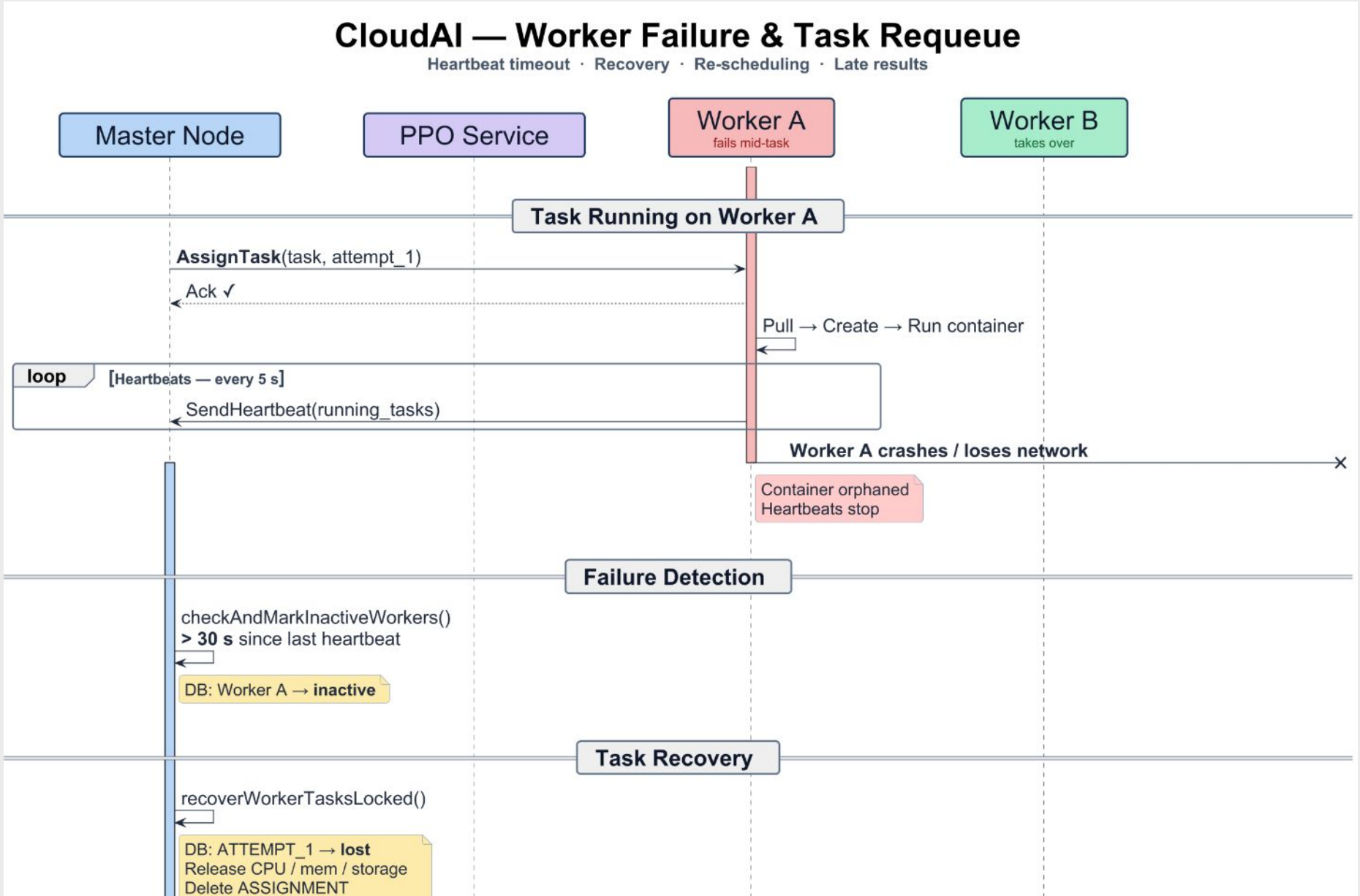


Task LifeCycle - Ideal

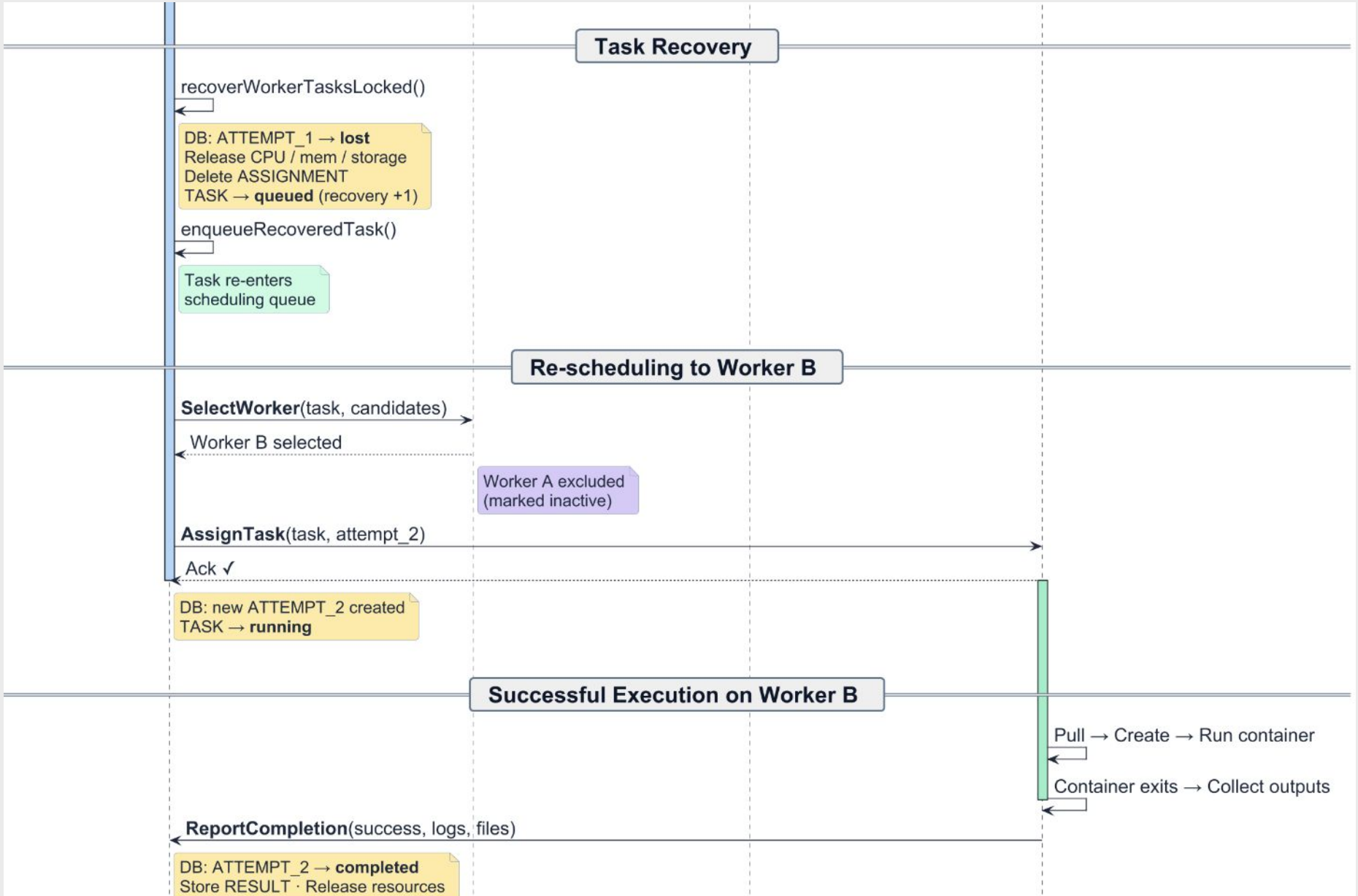




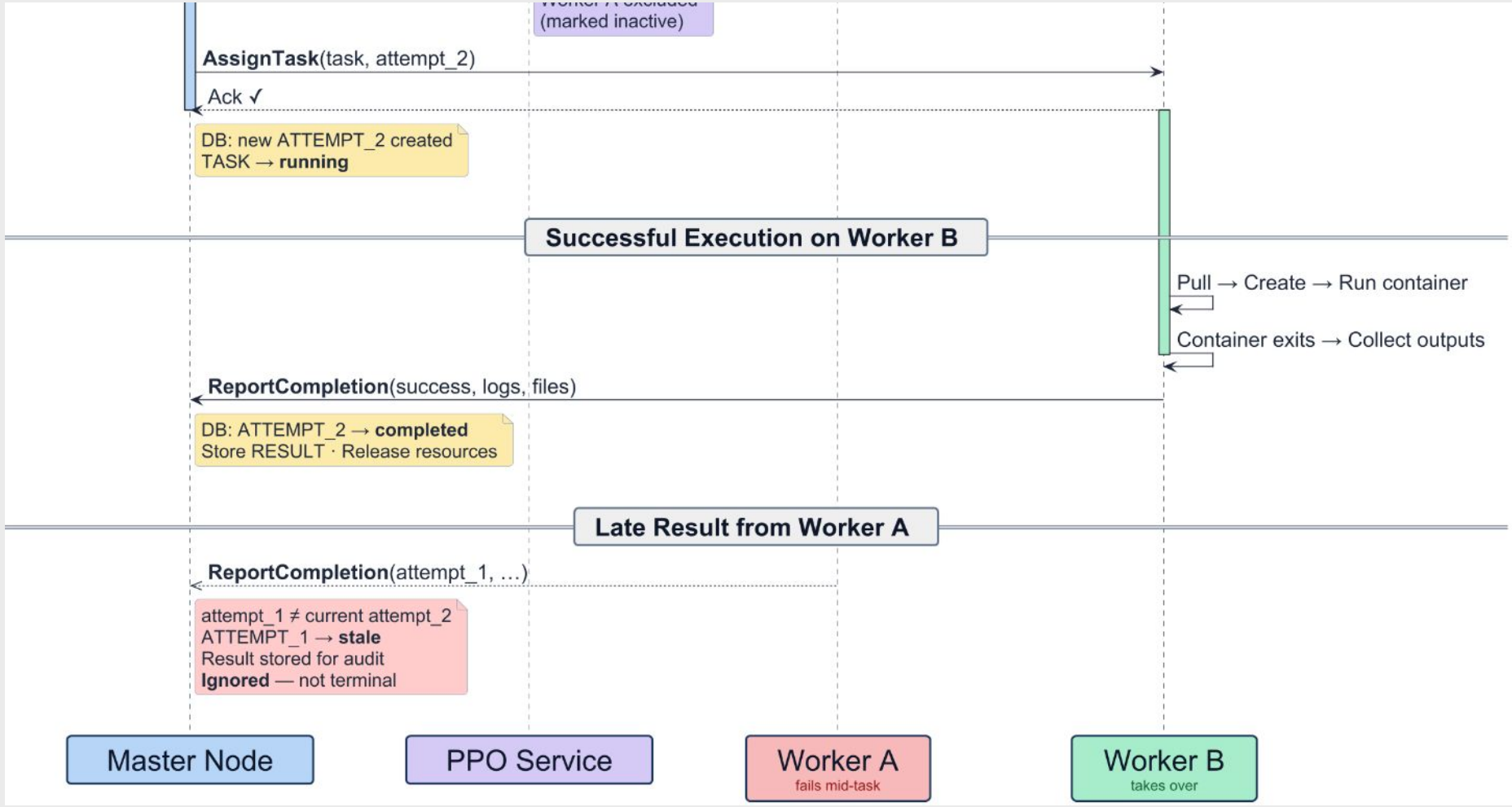
Task LifeCycle - Retry mechanism



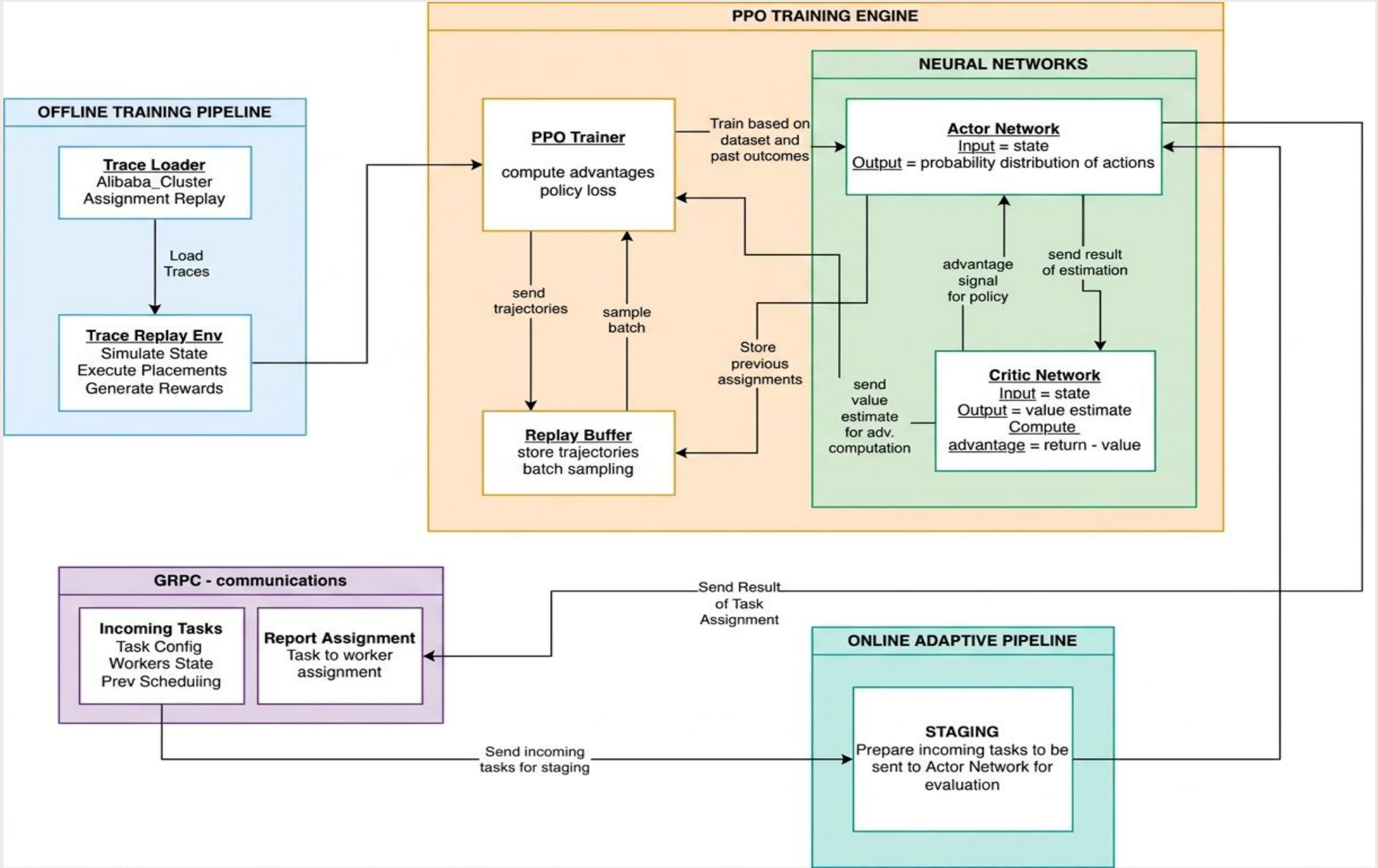
Task LifeCycle - Retry mechanism



Task LifeCycle - Retry mechanism



High-Level Architecture: Agentic Scheduler



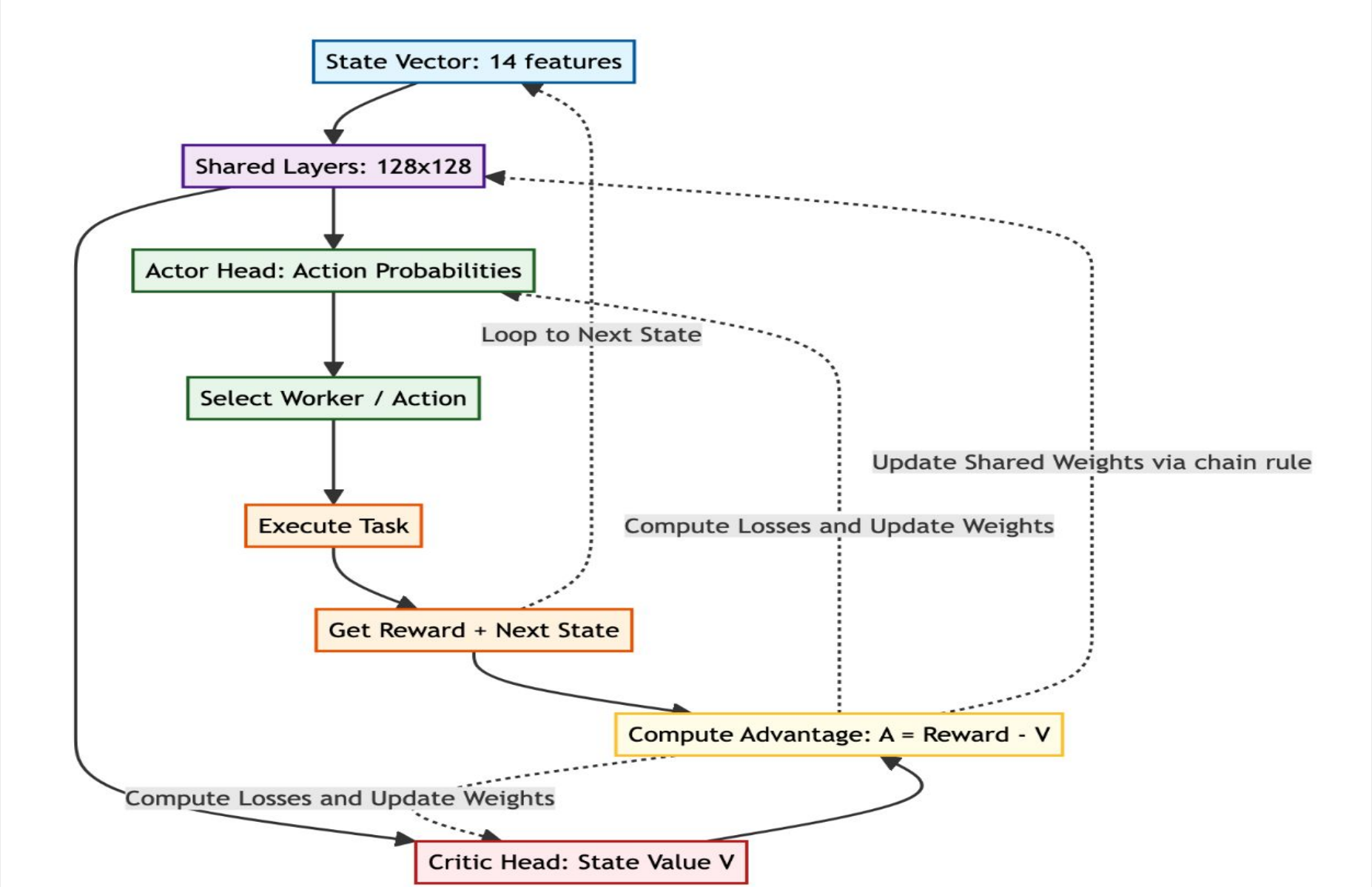
Methodology: Trace-Driven Training

- Trained using the Alibaba Cluster Trace v2018 (Production workload data).
- Replay Environment: Simulates sequential task arrivals, resource reservations, and completions.
- Lifecycle Resource Tracking: Matches actual simulated runtimes rather than inaccurate exponential decay.

State Representation & Action Masking

- Task Vector (5 features): CPU, Memory, Storage, SLA Multiplier, Task Type.
- Worker Vector (9 features): Available, Total, and Used ratios for CPU/Memory/Storage.
- Action Masking: Infeasible workers lacking required resources are mathematically hidden from the policy.

PPO Scheduler Design (Actor-Critic)

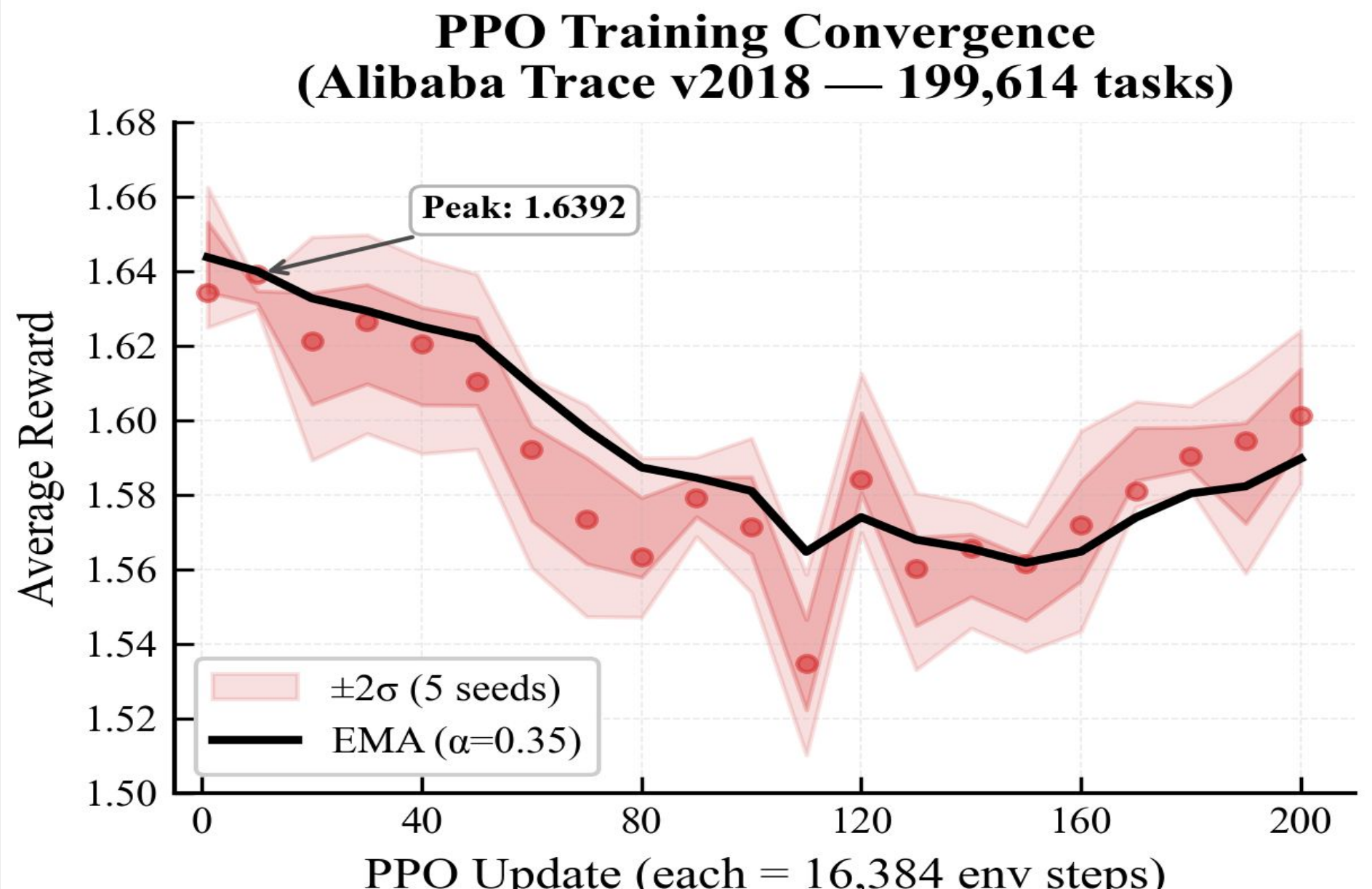


The Reward Function

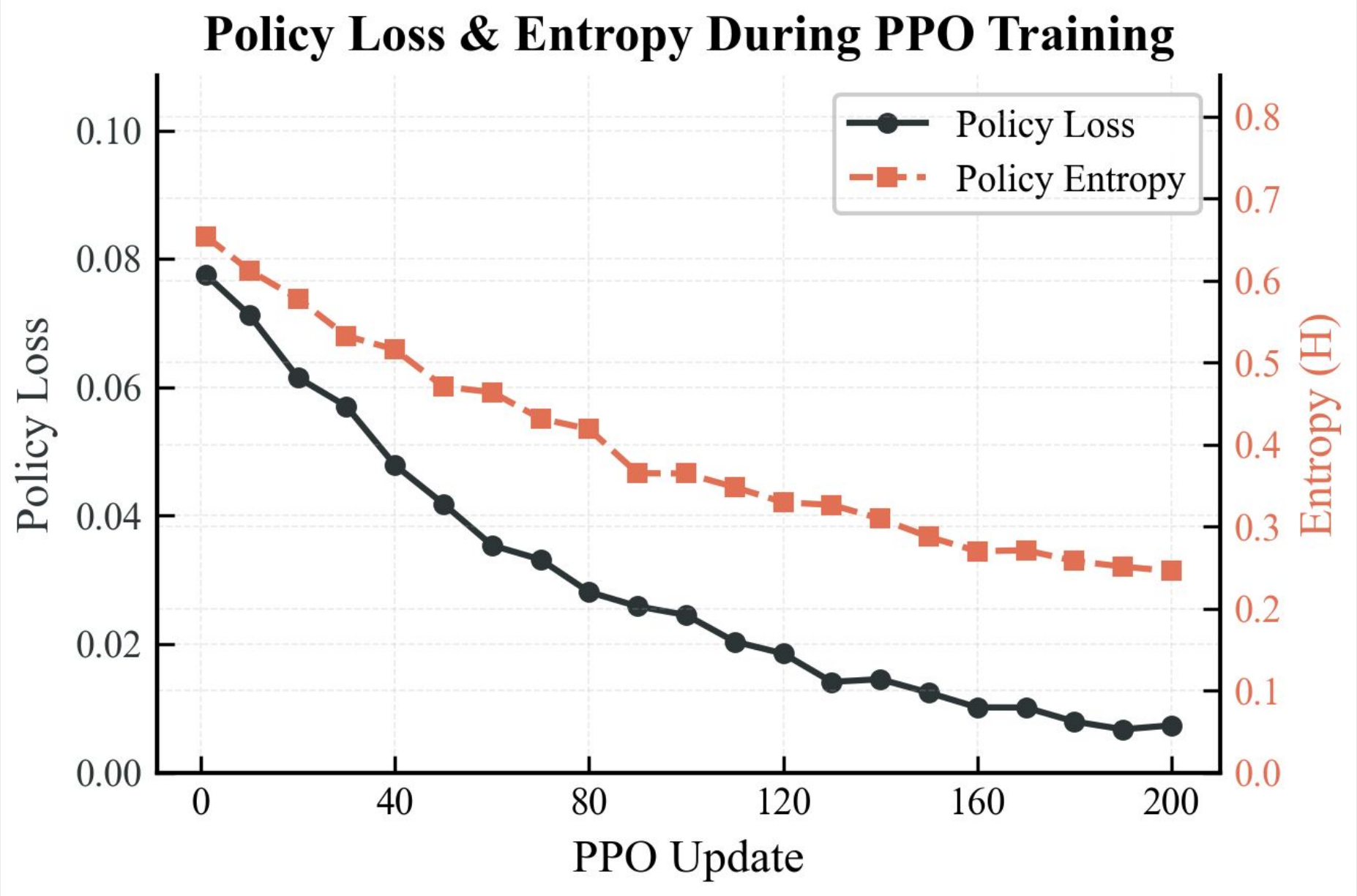
$$R = 1.4 + 0.25H - 0.35Q_p - 0.55T_p - 0.20I_p - 0.40\Delta I - R_q$$

- H is headroom bonus. It is rewarded by the selection of workers who have space vacant.
- Q_p is queue pressure. It punishes anticipated waiting.
- T_p is tail pressure. It penalizes likely SLA or tail-latency risk.
- I_p is imbalance penalty. It does not encourage overworking of already overworked workers.
- ΔI is change in imbalance. It punishes those behaviors that worsen cluster balance.
- R_q is requeue penalty. It discourages constantly dangerous placements.

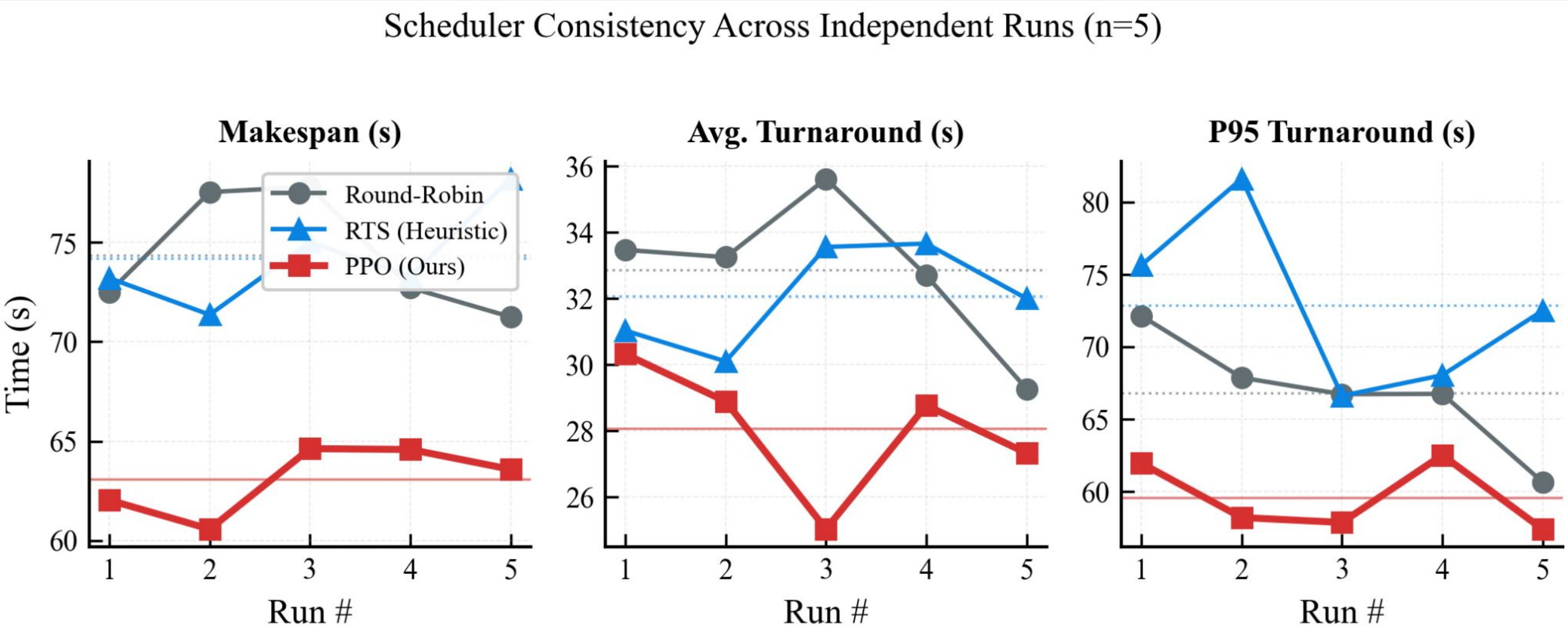
Training Convergence



Training Convergence



Results: KPI Benchmarking



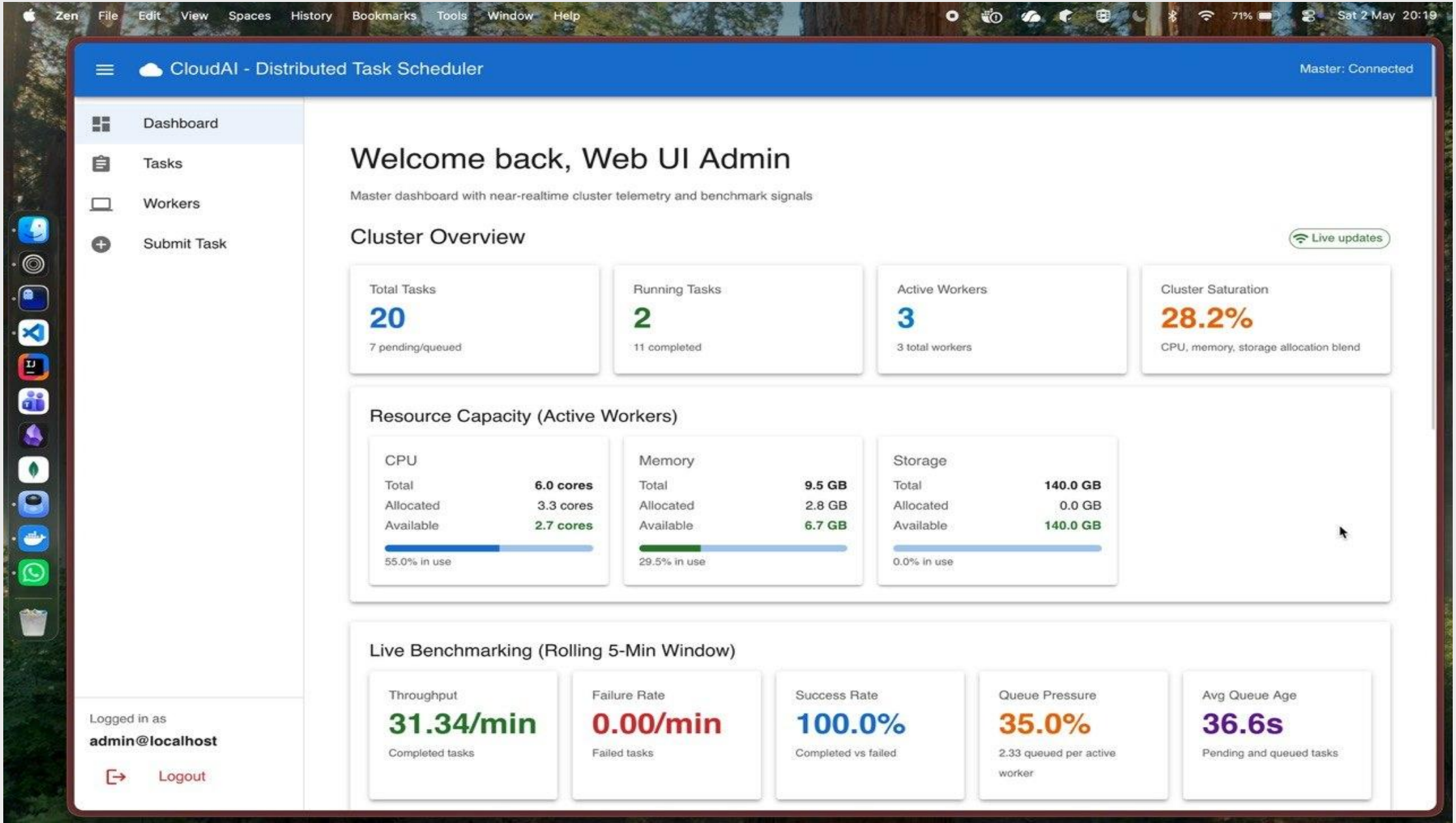
Pressure Scenarios & Cumulative Completion

Under bursty traffic:

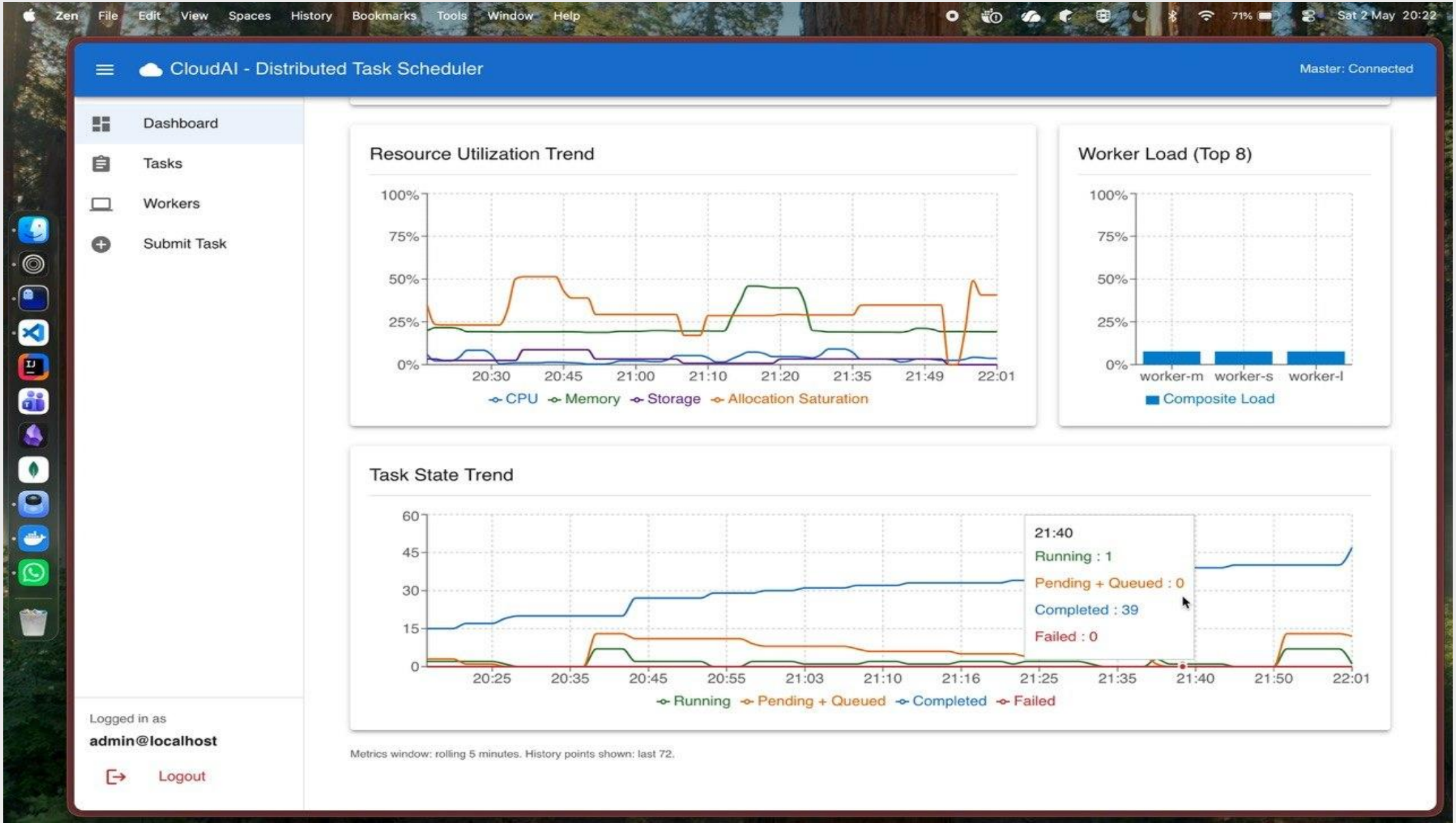
Comparison	Duration Improvement	Turnaround Improvement
PPO vs RR	8.7% faster	17.9% lower
PPO vs RTS	16.1% faster	16.9% lower

Scheduler	Tasks	Completed	Failed	Duration	Turnaround	P95
RR	20	20	0	70.06 s	33.80 s	69.00 s
RTS	20	20	0	76.20 s	33.40 s	74.00 s
PPO	20	20	0	**63.95 s**	**27.75 s**	**62.00 s**

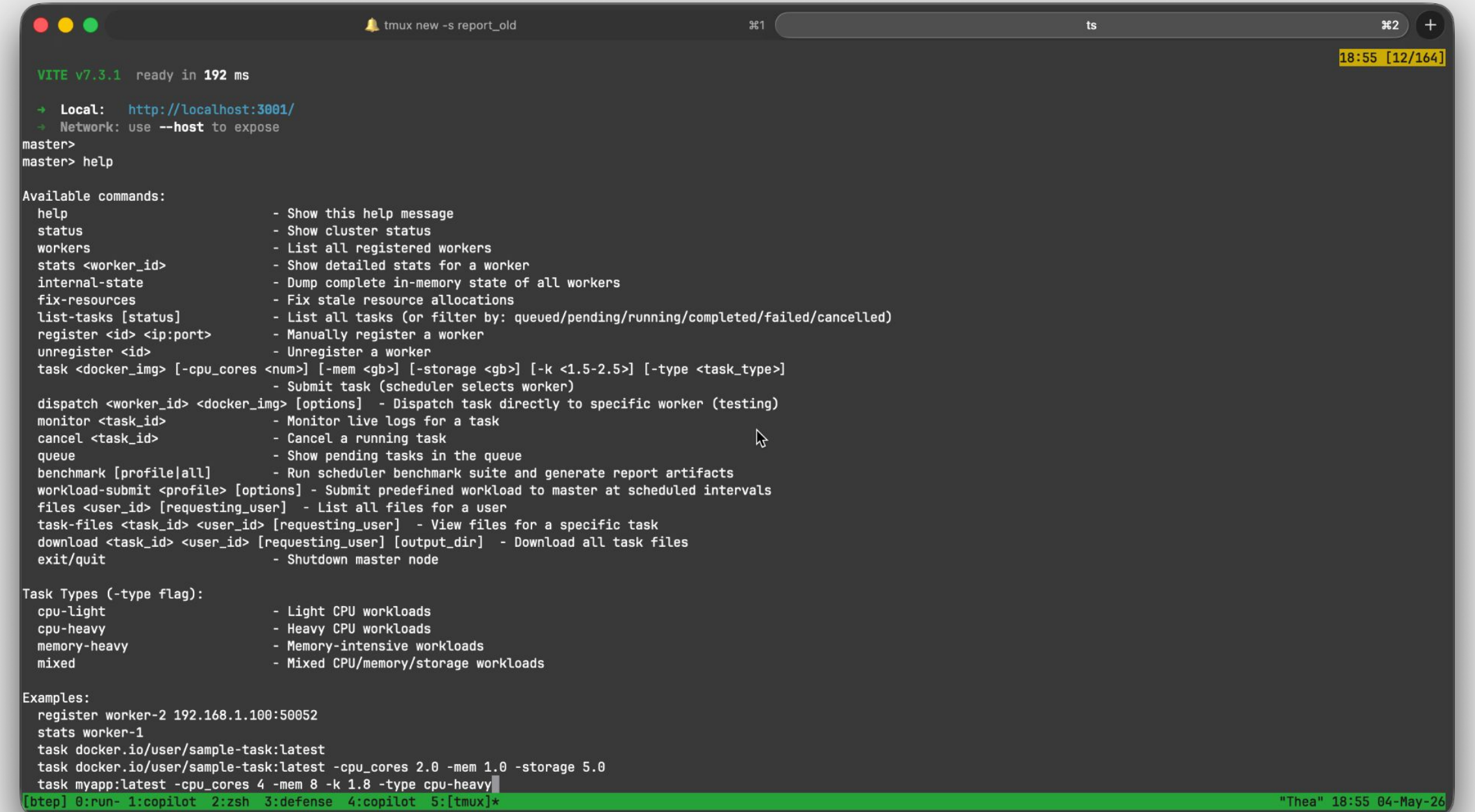
Project Outcomes - Web UI



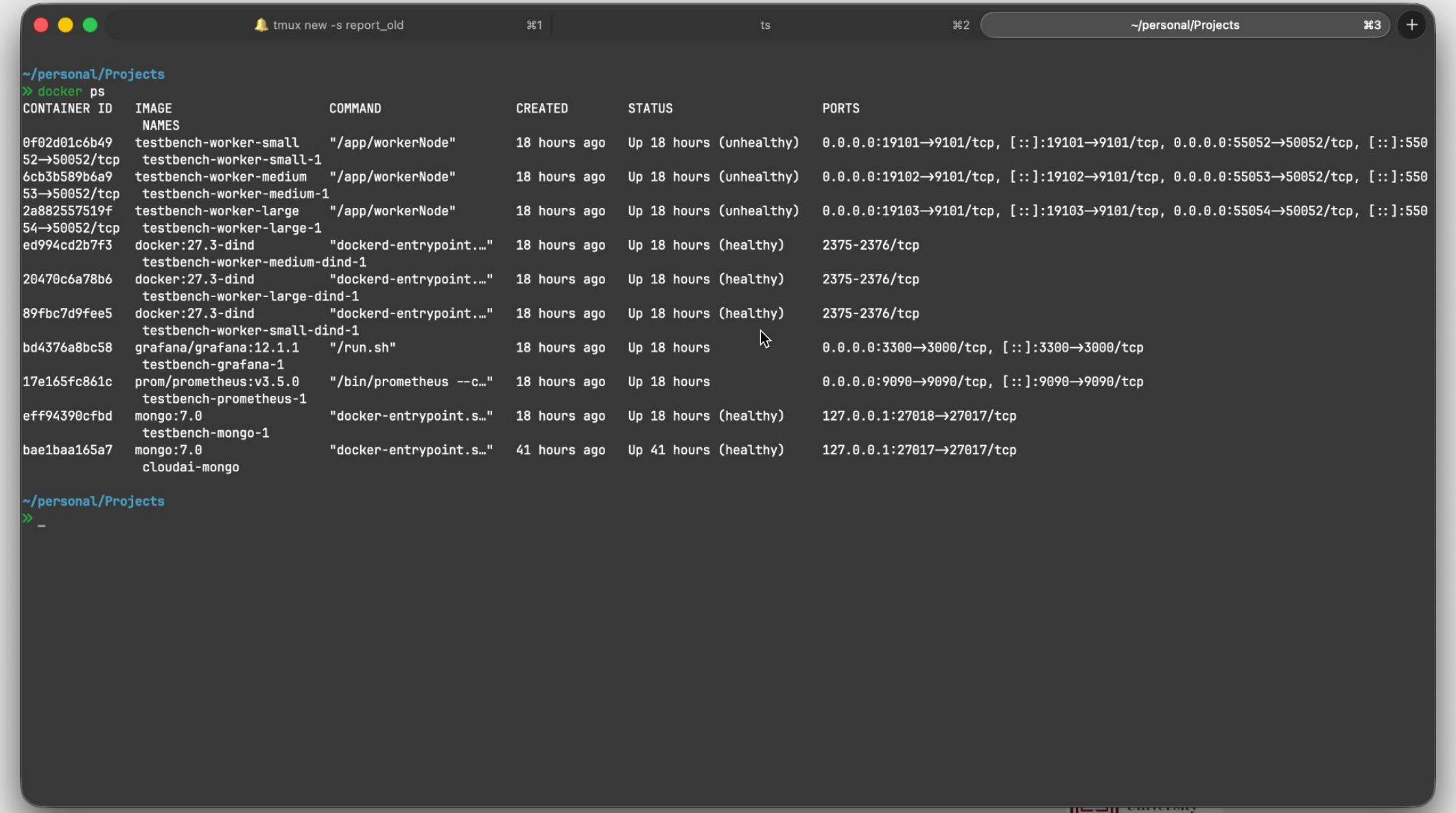
Project Outcomes - Live Plots



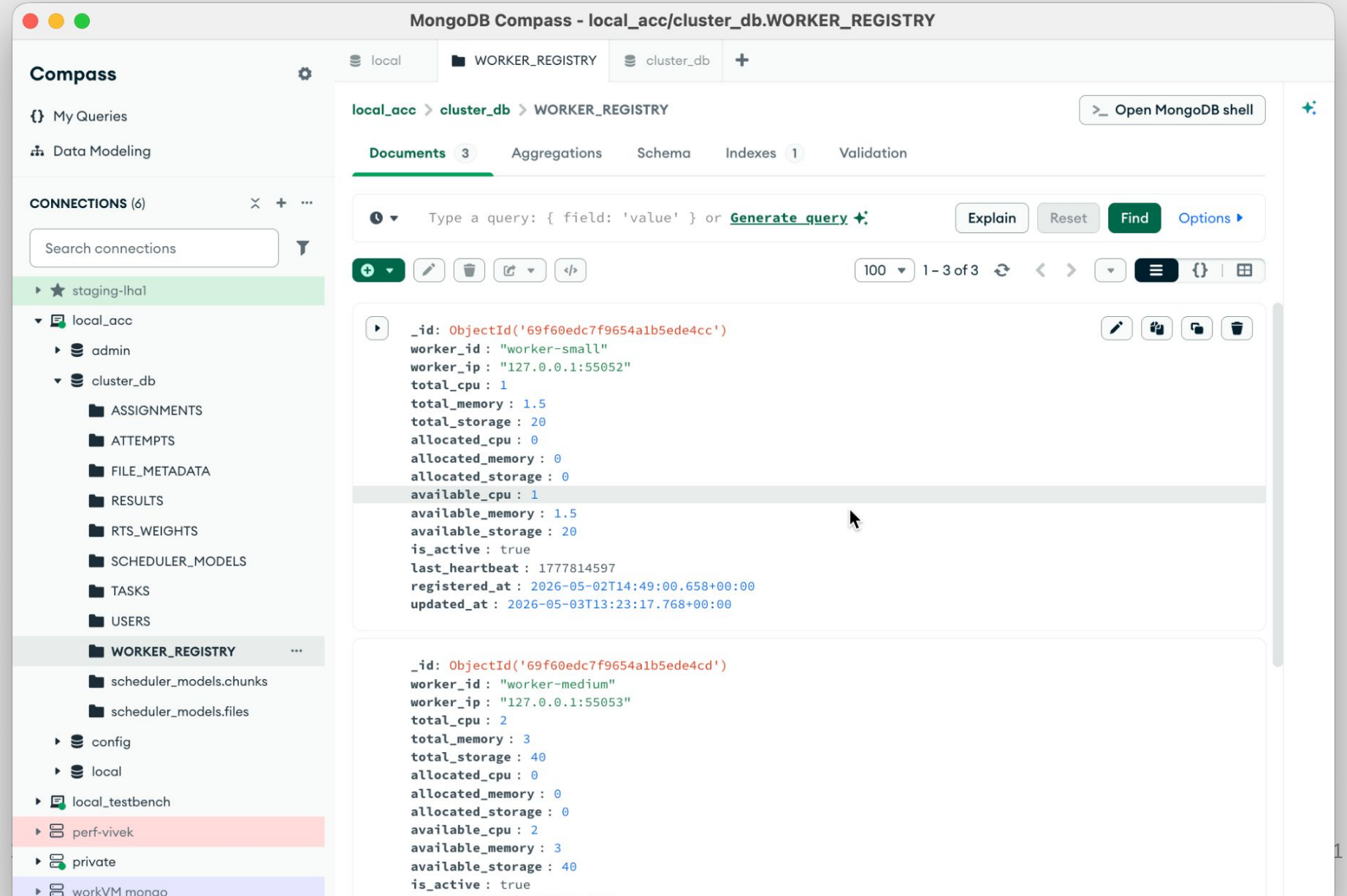
Project Outcomes - MasterNode CLI



Project Outcomes - Docker machines



Project Outcomes - DB & Worker Registry



Novelty in the project / Your contributions in the project

- Designed and built the Go master-worker framework from scratch.
- Engineered the clean scheduler integration boundary and gRPC communications.
- Developed the PPO training pipeline and trace replay environment.
- Implemented attempt-level recovery logic for fault tolerance.

Learning outcomes for you

- **Distributed Systems**: Learned the criticality of strict state handling, retries, and clean API boundaries.
- **Reinforcement Learning**: Realized the massive impact of reward shaping and safe deployment strategies.

Future Work - CRIU

- CRIU offers kernel-level support of freezing process state, dumping it to disk, and restoring it on demand.
- Benefits:
  - Container Migration
  - Task Preemption
  - Checkpointing failed workers

https://github.com/checkpoint-restore/criu26-02-23

Gantt chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17
Problem study and literature review	X	X	X														
Go master-worker framework		X	X	X	X												
Worker execution, heartbeats, and persistence				X	X	X	X	X									
Baseline schedulers and scheduler interface						X	X	X	X								
PPO training pipeline and trace replay								X	X	X	X	X					
PPO integration with Go control plane											X	X	X				
Benchmark campaigns and result analysis													X	X	X	X	
Report writing and final polishing																	X

Who will benefit from your work

1. Companies — Better handling of burstier traffic

2. Developers — Automate scheduling, fewer manual deployments

3. Users — Faster, more reliable services

Bibliography

- Gu, Y., Liu, Z., Dai, S., Liu, C., Wang, Y., Wang, S., Theodoropoulos, G., & Cheng, L. (2025, January 2). Deep Reinforcement Learning for job scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review. arXiv.org. https://arxiv.org/abs/2501.01007

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, July 20). Proximal Policy optimization Algorithms. arXiv.org. https://arxiv.org/abs/1707.06347

- Alibaba, "Alibaba cluster data." https://github.com/alibaba/clusterdata, 2018. Cluster traces released for cluster management research.

- https://github.com/checkpoint-restore/criu

Thank You!